

Experiment 'p_14_1_selection_sort_desc_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_14_1_selection_sort_desc_array

Natural language statement: null

Method signature: p_14_1_selection_sort_desc_array(inputs: array<int>) returns (sorted: array<int>)

Ensures

- forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] >= sorted[j]
- multiset(inputs[..]) == multiset(sorted[..])

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_14_1_selection_sort_desc_array(inputs: array<int>) returns (sorted: array<int>)`

The method should respect the following contract:

`ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] >= sorted[j], ensures`
 ↪ `multiset(inputs[..]) == multiset(sorted[..])`

Produce and show only the Dafny body of this method, including the curly braces that
 ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
 ↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPczkKoTfcvgWGy3LctJj6XOB0gr

Model: gpt-5.5-2026-04-23

Created at: 1780668957

Finish reason: stop

Usage: 310 tokens in, and 9827 tokens out

```
{
  sorted := new int[inputs.Length](k => inputs[k]);
  assert forall k :: 0 <= k < inputs.Length ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var n := sorted.Length;
  var i := 0;
  while i < n
    invariant sorted.Length == n
    invariant inputs.Length == n
    invariant 0 <= i <= n
    invariant forall a, b :: 0 <= a < b < i ==> sorted[a] >= sorted[b]
    invariant forall a, b :: 0 <= a < i <= b < n ==> sorted[a] >= sorted[b]
    invariant multiset(sorted[..]) == multiset(inputs[..])
    decreases n - i
  {
    var maxIdx := i;
    var j := i + 1;
    while j < n
```

```

invariant sorted.Length == n
invariant inputs.Length == n
invariant i < n
invariant i <= maxIdx < j <= n
invariant forall k :: i <= k < j ==> sorted[maxIdx] >= sorted[k]
invariant forall a, b :: 0 <= a < b < i ==> sorted[a] >= sorted[b]
invariant forall a, b :: 0 <= a < i <= b < n ==> sorted[a] >= sorted[b]
invariant multiset(sorted[..]) == multiset(inputs[..])
decreases n - j
{
  if sorted[j] > sorted[maxIdx] {
    maxIdx := j;
  }
  j := j + 1;
}

assert j == n;
ghost var prev := sorted[..];
assert |prev| == n;
assert forall k :: i <= k < n ==> prev[maxIdx] >= prev[k];
assert forall a, b :: 0 <= a < b < i ==> prev[a] >= prev[b];
assert forall a, b :: 0 <= a < i <= b < n ==> prev[a] >= prev[b];

var tmp := sorted[i];
sorted[i] := sorted[maxIdx];
sorted[maxIdx] := tmp;

assert sorted[i] == prev[maxIdx];
assert sorted[maxIdx] == prev[i];
assert forall k :: 0 <= k < n && k != i && k != maxIdx ==> sorted[k] == prev[k];

forall k | 0 <= k < n
  ensures sorted[k] == (prev[i := prev[maxIdx]] [maxIdx := prev[i]])[k]
{
  if k == i {
    if maxIdx == i {
      assert (prev[i := prev[maxIdx]] [maxIdx := prev[i]])[k] == prev[i];
    } else {
      assert (prev[i := prev[maxIdx]] [maxIdx := prev[i]])[k] == prev[maxIdx];
    }
  } else if k == maxIdx {
    assert (prev[i := prev[maxIdx]] [maxIdx := prev[i]])[k] == prev[i];
  } else {
    assert (prev[i := prev[maxIdx]] [maxIdx := prev[i]])[k] == prev[k];
  }
}
assert |sorted[..]| == |prev[i := prev[maxIdx]] [maxIdx := prev[i]]|;
assert sorted[..] == prev[i := prev[maxIdx]] [maxIdx := prev[i]];

if i == maxIdx {
  assert prev[i := prev[maxIdx]] [maxIdx := prev[i]] == prev;
}

```

```

} else {
  assert multiset(prev[i := prev[maxIdx]]) == multiset(prev) - multiset{prev[i]} +
    ↪ multiset{prev[maxIdx]};
  assert (prev[i := prev[maxIdx]])[maxIdx] == prev[maxIdx];
  assert multiset(prev[i := prev[maxIdx]][maxIdx := prev[i]]) ==
    multiset(prev[i := prev[maxIdx]]) - multiset{prev[maxIdx]} + multiset{prev[i
    ↪ ]};
  assert multiset(prev[i := prev[maxIdx]][maxIdx := prev[i]]) == multiset(prev);
}
assert multiset(sorted[..]) == multiset(prev);
assert multiset(sorted[..]) == multiset(inputs[..]);

var next := i + 1;

forall a, b | 0 <= a < b < next
  ensures sorted[a] >= sorted[b]
{
  if b < i {
    assert sorted[a] == prev[a];
    assert sorted[b] == prev[b];
    assert prev[a] >= prev[b];
  } else {
    assert b == i;
    assert sorted[a] == prev[a];
    assert sorted[b] == prev[maxIdx];
    assert prev[a] >= prev[maxIdx];
  }
}

forall a, b | 0 <= a < next <= b < n
  ensures sorted[a] >= sorted[b]
{
  if a < i {
    assert sorted[a] == prev[a];
    if b == maxIdx {
      assert sorted[b] == prev[i];
      assert prev[a] >= prev[i];
    } else {
      assert sorted[b] == prev[b];
      assert prev[a] >= prev[b];
    }
  } else {
    assert a == i;
    assert sorted[a] == prev[maxIdx];
    if b == maxIdx {
      assert sorted[b] == prev[i];
      assert prev[maxIdx] >= prev[i];
    } else {
      assert sorted[b] == prev[b];
      assert prev[maxIdx] >= prev[b];
    }
  }
}

```

```

    }
  }

  i := next;
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_1_selection_sort_desc_array/program.dfy(5,46): Error: index out of
  ↳ range
|
5 | sorted := new int[inputs.Length](k => inputs[k]);
  | ^

problem_data/p_14_1_selection_sort_desc_array/program.dfy(5,46): Error: insufficient
  ↳ reads clause to read array element; Consider extracting inputs[k] to a local
  ↳ variable before the lambda expression, or adding 'reads inputs' in the enclosing
  ↳ lambda specification for resolution
|
5 | sorted := new int[inputs.Length](k => inputs[k]);
  | ^

```

Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPgg7Y34IWBKND05NeUr8R2c9X2

Model: gpt-5.5-2026-04-23

Created at: 1780669186

Finish reason: stop

Usage: 1935 tokens in, and 6376 tokens out

```

{
  var s := inputs[..];
  var n := |s|;

  var i := 0;
  while i < n
    invariant |s| == n
    invariant n == inputs.Length
    invariant 0 <= i <= n
    invariant forall a, b :: 0 <= a < b < i ==> s[a] >= s[b]

```

```

invariant forall a, b :: 0 <= a < i <= b < n ==> s[a] >= s[b]
invariant multiset(s) == multiset(inputs[..])
decreases n - i
{
  var maxIdx := i;
  var j := i + 1;
  while j < n
    invariant |s| == n
    invariant n == inputs.Length
    invariant 0 <= i < n
    invariant i <= maxIdx < j <= n
    invariant forall k :: i <= k < j ==> s[maxIdx] >= s[k]
    invariant forall a, b :: 0 <= a < b < i ==> s[a] >= s[b]
    invariant forall a, b :: 0 <= a < i <= b < n ==> s[a] >= s[b]
    invariant multiset(s) == multiset(inputs[..])
    decreases n - j
    {
      if s[j] > s[maxIdx] {
        maxIdx := j;
      }
      j := j + 1;
    }
}

assert j == n;
var prev := s;
var vi := prev[i];
var vmax := prev[maxIdx];
assert forall k :: i <= k < n ==> prev[maxIdx] >= prev[k];

s := prev[i := vmax][maxIdx := vi];
assert |s| == n;

if maxIdx == i {
  assert vi == vmax;
  assert prev[i := vmax] == prev;
  assert s == prev[i := vi];
  assert prev[i := vi] == prev;
  assert s == prev;
} else {
  assert i < maxIdx;
  var mid := prev[i := vmax];
  assert s == mid[maxIdx := vi];
  assert mid[maxIdx] == vmax;

  if vi == vmax {
    assert mid == prev;
    assert s == prev[maxIdx := vi];
    assert prev[maxIdx := vi] == prev;
    assert s == prev;
  } else {
    assert multiset(mid) == multiset(prev) - multiset{vi} + multiset{vmax};
  }
}

```

```

    assert multiset(s) == multiset(mid) - multiset{vmax} + multiset{vi};
    assert multiset(s) == multiset(prev);
}

assert s[i] == prev[maxIdx];
assert s[maxIdx] == prev[i];
forall k | 0 <= k < n && k != i && k != maxIdx
    ensures s[k] == prev[k]
{
    assert mid[k] == prev[k];
    assert s[k] == mid[k];
}
}

assert s[i] == prev[maxIdx];
assert s[maxIdx] == prev[i];
assert forall k :: 0 <= k < n && k != i && k != maxIdx ==> s[k] == prev[k];
assert multiset(s) == multiset(prev);
assert multiset(s) == multiset(inputs[..]);

var next := i + 1;

forall a, b | 0 <= a < b < next
    ensures s[a] >= s[b]
{
    if b < i {
        assert a != i && a != maxIdx;
        assert b != i && b != maxIdx;
        assert s[a] == prev[a];
        assert s[b] == prev[b];
        assert prev[a] >= prev[b];
    } else {
        assert b == i;
        assert a < i;
        assert a != i && a != maxIdx;
        assert s[a] == prev[a];
        assert s[b] == prev[maxIdx];
        assert prev[a] >= prev[maxIdx];
    }
}

forall a, b | 0 <= a < next <= b < n
    ensures s[a] >= s[b]
{
    if a < i {
        assert a != i && a != maxIdx;
        assert s[a] == prev[a];
        if b == maxIdx {
            assert s[b] == prev[i];
            assert prev[a] >= prev[i];
        } else {

```

```

        assert b != i;
        assert s[b] == prev[b];
        assert prev[a] >= prev[b];
    }
} else {
    assert a == i;
    assert s[a] == prev[maxIdx];
    if b == maxIdx {
        assert s[b] == prev[i];
        assert prev[maxIdx] >= prev[i];
    } else {
        assert b != i;
        assert s[b] == prev[b];
        assert prev[maxIdx] >= prev[b];
    }
}
}

i := next;
}

assert i == n;
sorted := new int[n];
assert sorted.Length == n;
assert sorted != inputs;

var k := 0;
while k < n
    invariant sorted.Length == n
    invariant sorted != inputs
    invariant |s| == n
    invariant i == n
    invariant 0 <= k <= n
    invariant forall t :: 0 <= t < k ==> sorted[t] == s[t]
    invariant forall a, b :: 0 <= a < b < n ==> s[a] >= s[b]
    invariant multiset(s) == multiset(inputs[..])
    decreases n - k
{
    sorted[k] := s[k];
    k := k + 1;
}

assert forall t :: 0 <= t < n ==> sorted[t] == s[t];
assert |sorted[..]| == |s|;
forall t | 0 <= t < |s|
    ensures sorted[..][t] == s[t]
{
    assert sorted[..][t] == sorted[t];
    assert sorted[t] == s[t];
}
assert sorted[..] == s;

```



```

forall a, b | 0 <= a < b < sorted.Length
  ensures sorted[a] >= sorted[b]
{
  assert sorted.Length == n;
  assert sorted[a] == s[a];
  assert sorted[b] == s[b];
  assert s[a] >= s[b];
}

assert multiset(sorted[..]) == multiset(s);
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Final Program

```

method p_14_1_selection_sort_desc_array(inputs: array<int>) returns (sorted: array<int>)
  ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] >= sorted[j]
  ensures multiset(inputs[..]) == multiset(sorted[..])
{
  var s := inputs[..];
  var n := |s|;

  var i := 0;
  while i < n
    invariant |s| == n
    invariant n == inputs.Length
    invariant 0 <= i <= n
    invariant forall a, b :: 0 <= a < b < i ==> s[a] >= s[b]
    invariant forall a, b :: 0 <= a < i <= b < n ==> s[a] >= s[b]
    invariant multiset(s) == multiset(inputs[..])
    decreases n - i
  {
    var maxIdx := i;
    var j := i + 1;
    while j < n
      invariant |s| == n
      invariant n == inputs.Length
      invariant 0 <= i < n
      invariant i <= maxIdx < j <= n
      invariant forall k :: i <= k < j ==> s[maxIdx] >= s[k]
      invariant forall a, b :: 0 <= a < b < i ==> s[a] >= s[b]
      invariant forall a, b :: 0 <= a < i <= b < n ==> s[a] >= s[b]
      invariant multiset(s) == multiset(inputs[..])
      decreases n - j
    {
      if s[j] > s[maxIdx] {
        maxIdx := j;
      }
      j := j + 1;
    }
  }
}

```

```

}

assert j == n;
var prev := s;
var vi := prev[i];
var vmax := prev[maxIdx];
assert forall k :: i <= k < n ==> prev[maxIdx] >= prev[k];

s := prev[i := vmax][maxIdx := vi];
assert |s| == n;

if maxIdx == i {
  assert vi == vmax;
  assert prev[i := vmax] == prev;
  assert s == prev[i := vi];
  assert prev[i := vi] == prev;
  assert s == prev;
} else {
  assert i < maxIdx;
  var mid := prev[i := vmax];
  assert s == mid[maxIdx := vi];
  assert mid[maxIdx] == vmax;

  if vi == vmax {
    assert mid == prev;
    assert s == prev[maxIdx := vi];
    assert prev[maxIdx := vi] == prev;
    assert s == prev;
  } else {
    assert multiset(mid) == multiset(prev) - multiset{vi} + multiset{vmax};
    assert multiset(s) == multiset(mid) - multiset{vmax} + multiset{vi};
    assert multiset(s) == multiset(prev);
  }

  assert s[i] == prev[maxIdx];
  assert s[maxIdx] == prev[i];
  forall k | 0 <= k < n && k != i && k != maxIdx
    ensures s[k] == prev[k]
  {
    assert mid[k] == prev[k];
    assert s[k] == mid[k];
  }
}

assert s[i] == prev[maxIdx];
assert s[maxIdx] == prev[i];
assert forall k :: 0 <= k < n && k != i && k != maxIdx ==> s[k] == prev[k];
assert multiset(s) == multiset(prev);
assert multiset(s) == multiset(inputs[..]);

var next := i + 1;

```

```

forall a, b | 0 <= a < b < next
  ensures s[a] >= s[b]
{
  if b < i {
    assert a != i && a != maxIdx;
    assert b != i && b != maxIdx;
    assert s[a] == prev[a];
    assert s[b] == prev[b];
    assert prev[a] >= prev[b];
  } else {
    assert b == i;
    assert a < i;
    assert a != i && a != maxIdx;
    assert s[a] == prev[a];
    assert s[b] == prev[maxIdx];
    assert prev[a] >= prev[maxIdx];
  }
}

forall a, b | 0 <= a < next <= b < n
  ensures s[a] >= s[b]
{
  if a < i {
    assert a != i && a != maxIdx;
    assert s[a] == prev[a];
    if b == maxIdx {
      assert s[b] == prev[i];
      assert prev[a] >= prev[i];
    } else {
      assert b != i;
      assert s[b] == prev[b];
      assert prev[a] >= prev[b];
    }
  } else {
    assert a == i;
    assert s[a] == prev[maxIdx];
    if b == maxIdx {
      assert s[b] == prev[i];
      assert prev[maxIdx] >= prev[i];
    } else {
      assert b != i;
      assert s[b] == prev[b];
      assert prev[maxIdx] >= prev[b];
    }
  }
}

i := next;
}

```

```

assert i == n;
sorted := new int[n];
assert sorted.Length == n;
assert sorted != inputs;

var k := 0;
while k < n
  invariant sorted.Length == n
  invariant sorted != inputs
  invariant |s| == n
  invariant i == n
  invariant 0 <= k <= n
  invariant forall t :: 0 <= t < k ==> sorted[t] == s[t]
  invariant forall a, b :: 0 <= a < b < n ==> s[a] >= s[b]
  invariant multiset(s) == multiset(inputs[..])
  decreases n - k
{
  sorted[k] := s[k];
  k := k + 1;
}

assert forall t :: 0 <= t < n ==> sorted[t] == s[t];
assert |sorted[..]| == |s|;
forall t | 0 <= t < |s|
  ensures sorted[..][t] == s[t]
{
  assert sorted[..][t] == sorted[t];
  assert sorted[t] == s[t];
}
assert sorted[..] == s;

forall a, b | 0 <= a < b < sorted.Length
  ensures sorted[a] >= sorted[b]
{
  assert sorted.Length == n;
  assert sorted[a] == s[a];
  assert sorted[b] == s[b];
  assert s[a] >= s[b];
}

assert multiset(sorted[..]) == multiset(s);
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Total Token Usage

Input tokens: 2245

Output tokens: 16203

Reasoning tokens: 13130

Sum of 'total tokens': 18448

Experiment Timings

Overall Experiment started at 1780668956393, ended at 1780669345399, lasting 389006ms (389.01 seconds)

Iteration #1 started at 1780668956395, ended at 1780669185759, lasting 229364ms (229.36 seconds)

Iteration #2 started at 1780669185812, ended at 1780669345399, lasting 159587ms (159.59 seconds)